Itnavideo — YouTube Subtitles Generator: Audit & Implementation Report

The specification is a solid foundation for a focused, long-form subtitle-burning tool, but it is currently **under-specified for production**: it lacks a complete caption-editing model, validation and recovery flows, accessibility requirements, mobile behavior, render observability, and precise business/credit rules. The most important improvement is to make the product **stateful, resumable, and trustworthy**: users must be able to save work, correct transcription reliably, preview accurately, and recover from failures without losing money or work. Remotion Lambda supports chunked rendering, retries, and progress polling, but the spec does not define how these capabilities will be used.

Executive assessment

Area	Current state	Severity	Primary recommendation
User workflow	Linear 4-step flow, but no save/resume, project naming, or versioning	High	Add persistent drafts, autosave, and resume-from-step
Transcript editing	“Editorial transcript” is mentioned, but editing model is undefined	Critical	Define segment-level editing, speaker labels, timing preservation, and diff/re-render

Caption styling	32 presets and basic sizing/casing; no contrast, positioning, or readability controls	High	Add validated style controls, safe-zone visualization, and accessibility contrast checks
Preview	“Live Cinema Preview” is vague; no playback, scrubbing, or WYSIWYG guarantee	Critical	Build a frame-accurate preview with audio, captions, and safe-zone overlay
Rendering	Lambda render is defined, but job lifecycle, progress, retries, and delivery are incomplete	Critical	Add job state machine, idempotency, progress, cancellation, and artifact lifecycle
Errors	Only three edge cases are covered	High	Define a complete error taxonomy, user recovery paths, and observability
Mobile	No mobile-specific requirements despite a responsive layout claim	High	Create a mobile-first studio layout with progressive disclosure
Accessibility	No accessibility requirements	High	Apply WCAG 2.2 AA to studio UI and caption output

QA	Three narrow tests only	High	Add E2E, visual, performance, accessibility, security, and failure-injection tests

Critical product gaps

1. Define the caption data model

The spec says users can correct typos and Hinglish spellings, but it does not define the underlying data structure or what happens when text changes. This is the single largest functional gap because transcription, word highlighting, rendering, and re-rendering all depend on it.

Required model:

- Store transcription as an ordered array of caption segments.
- Each segment must contain: unique ID, start time, end time, text, confidence score, speaker label where available, language/locale, and word-level timing data.
- Preserve original ASR text separately from user-edited text so the product can show a “reviewed” state and support undo/redo.
- Support word-level timing because the spec promises word highlighting within one frame, or approximately 33 ms at 30 FPS.
- Define behavior when a user edits a word: retain the original timing unless the edit materially changes duration; allow the user to nudge segment start/end times.
- Add a transcript validation state: `raw`, `reviewing`, `approved`, and `locked_for_render`.
- Prevent render until the transcript has passed basic validation: non-empty text, monotonic timings, no overlapping segments beyond a defined tolerance, and no segment extending beyond media duration.

Implementation instruction for Antigravity: introduce a canonical `SubtitleProject` state object shared by the studio, preview, and render payload. Persist it server-side after every meaningful change, with optimistic UI updates and debounced

autosave. Do not derive caption state independently in the preview and Remotion composition; both must consume the same normalized data contract.

2. Add draft persistence and resume

The current flow implies that a user must complete upload, transcription, styling, and rendering in one session. That is fragile for a five-minute video workflow and creates abandonment risk.

Required capabilities:

- Create a draft immediately after a successful upload.
- Autosave transcript edits, selected preset, safe-zone mode, music selection, volume, font size, casing, and render preferences.
- Show a clear “Draft saved” indicator, last-saved timestamp, and explicit saving/error state.
- Let users return to `/dashboard/youtube-subtitles` and resume an incomplete project.
- Maintain a project list with name, thumbnail or media type, duration, status, last edited time, and primary action: `Continue`, `Preview`, `Download`, or `Duplicate`.
- Support duplicate project, rename, delete with confirmation, and archive.
- Define draft retention: for free users, retain drafts for at least 30 days; for paid users, retain according to plan. State the policy in the UI.

Why: Users will spend meaningful time correcting transcripts. Losing that work after refresh, tab closure, network interruption, or render failure would be unacceptable.

3. Clarify the actual output promise

The spec says the product “burns subtitles,” but it does not state whether it also produces an SRT/VTT file, whether the original video audio is preserved, or whether the output is a new MP4 with captions composited into the video.

Required clarification and features:

- State explicitly that the final deliverable is a caption-burned MP4.

- Offer an optional sidecar export: `.srt` and/or `.vtt`, generated from the same approved transcript.
- Offer an optional "captions-only" export for creators who want to upload YouTube captions separately.
- Preserve the source audio track by default; do not silently re-encode or remove it.
- State output codec, container, bitrate target, audio codec, and whether the output is variable or constant frame rate.
- Add a "Copy caption text" action and a downloadable transcript in plain text.
- If the uploaded source is audio-only, clearly state that the output will be either an audio waveform video or an explicitly chosen static visual; do not imply a full video output when only audio was uploaded.

Why: "YouTube Subtitles Generator" can mean burned-in captions, an SRT file, or both. Ambiguity will cause user disappointment and support tickets.

Workflow improvements

Current workflow issues

Step	Gap	Required improvement
Upload	No upload progress, cancel, retry, virus/format validation, or duration validation	Add resumable multipart upload, progress, cancel, retry, and preflight validation
Transcription	No language selection, speaker handling, confidence display, or editing recovery	Add language selection, confidence indicators, speaker labels, and editable transcript

Music	Music is optional but its interaction with source audio is unclear	Clarify mixing, ducking, fade in/out, and whether music is added to the final MP4
Style	32 presets are listed but not categorized, searchable, or previewable consistently	Add preset categories, search, favorites, and deterministic preview
Preview	No explicit playback controls or frame-accurate caption verification	Add timeline, scrubber, play/pause, frame stepping, and caption overlay
Render	No queue position, estimated time, cancellation, or failure recovery	Add render job tracking and transparent status

Recommended user workflow

Replace the current four-step flow with a six-stage, non-destructive workflow:

1. **Upload and validate**
2. **Transcribe and review**
3. **Style captions**
4. **Position and safe-zone check**
5. **Preview and export settings**
6. **Render, track, and download**

Each stage should be independently accessible after its prerequisites are complete. Users should be able to move backward without losing edits, but the UI must clearly indicate when a change invalidates a prior render or preview.

Upload experience

The current upload step needs substantially more detail.

Required behavior:

- Accept drag-and-drop, file picker, and potentially URL import only if licensing and download rights can be validated.
- Show file name, size, duration, resolution, frame rate where available, and estimated processing time.
- Validate before upload: accepted MIME types, file size, duration, audio presence, and aspect ratio.
- For a 16:9 requirement, do not reject non-16:9 media outright if cropping or letterboxing can be handled; instead, show a clear aspect-ratio warning and offer:
  - `Crop to 16:9`
  - `Letterbox / pillarbox`
  - `Cancel and replace media`
- Show a thumbnail preview and allow replacement before transcription begins.
- Support upload cancellation and retry.
- Use resumable uploads for large files, with progress percentage and network-loss recovery.
- Store uploaded media in durable object storage with signed, expiring access URLs; never expose long-lived credentials to the browser.

Implementation instruction for Antigravity: add a dedicated upload state machine: `idle`, `validating`, `uploading`, `processing`, `ready`, `failed`, `cancelled`. Persist upload progress and allow recovery after refresh. Block transcription until media validation succeeds.

Transcription and editing

“Live editorial transcript” is too vague to implement consistently.

Required transcript editor:

- Display transcript in readable caption blocks, synchronized with playback.
- Allow inline text editing per segment.

- Support split segment, merge adjacent segments, delete segment, and adjust start/end timing.
- Provide undo/redo with at least 25 steps.
- Highlight low-confidence words or segments using a non-color-only indicator, such as an underline, icon, and tooltip.
- Provide find-and-replace for recurring names, terms, and Hinglish spellings.
- Support speaker labels when diarization is available; otherwise, allow manual speaker assignment.
- Show word-level timings in an optional advanced panel, but do not overwhelm first-time users.
- Include a “Mark as reviewed” action that explicitly confirms the transcript before rendering.
- Preserve edits across refresh, browser restart, and render retries.

Important edge case: if a user edits text after transcription but before rendering, the render must use the edited transcript—not stale ASR output. Define this explicitly in the render payload.

Music and audio controls

The current music feature is under-defined and potentially confusing.

Required behavior:

- Make clear that background music is **optional** and that the default is muted.
- Show a waveform or simple audio-level visualization for the source audio.
- Add independent controls for:
    - Music volume
    - Source-audio volume
    - Fade-in duration
    - Fade-out duration
    - Music start offset
- Add optional auto-ducking: reduce music during detected speech.
- Clearly display the final audio mix in preview.
- Prevent users from selecting a music track if the source has no audio.
- Show licensing information beside every music track.
- Do not describe music as “AWS S3 background tracks” in user-facing UI; users care about the track, mood, duration, and license, not the storage provider.

Implementation instruction for Antigravity: separate `sourceAudio`, `musicTrack`, and `finalAudioMix` in project state. Ensure the Remotion composition receives explicit audio layer instructions rather than inferring behavior from a single volume value.

Caption style and readability

Preset system

The spec promises 32 animated typography presets but gives no taxonomy, preview contract, or accessibility constraints.

Required improvements:

- Group presets by use case: `Clean`, `Documentary`, `Educational`, `Bold`, `Cinematic`, `Accessible`, and `Trend-driven`.
- Add search and filters: font style, animation, background, color, readability, and popularity.
- Show a consistent preset thumbnail using a representative phrase, not random text.
- Support favorites and recently used presets.
- Preserve user style choices when switching presets where possible; do not silently reset font size, casing, or safe-zone mode.
- Add a “Reset to preset defaults” action.
- Avoid claiming presets are “authentic” to named creators unless licensing, permission, or a defensible design rationale exists. Use descriptive names such as `Clean Sans`, `Documentary Serif`, and `Lecture Focus` instead.

Typography controls

Current controls—four sizes and sentence case versus uppercase—are too limited for a professional subtitle tool.

Required controls:

- Font size: use a numeric control or slider with accessible labels, plus the existing named presets.
- Text color and background color, constrained to accessible combinations.
- Background style: none, translucent box, solid box, gradient, outline, and shadow.
- Text alignment: center, left, right.
- Position: bottom, lower third, middle, top.
- Line height and letter spacing, with sensible bounds.
- Maximum characters per line and maximum lines per caption.
- Animation intensity: none, subtle, standard, expressive.
- Emphasis: keyword highlighting, uppercase, bold, or none.
- Safe-zone offset, with visual feedback.

Accessibility requirement: caption text must meet at least 4.5:1 contrast against its background or backing treatment; UI controls and meaningful graphical elements should meet the applicable WCAG contrast requirements.

Implementation instruction for Antigravity: validate style combinations at input time. If a user selects low-contrast text, show a non-blocking warning and offer a one-click accessible alternative. Do not block rendering solely because of contrast, but make the risk visible.

Safe-zone system

The current safe-zone specification is too absolute. A fixed 84 px bottom margin may prevent collision with a web scrubber, but it does not account for varying UI densities, browser zoom, device pixel ratios, or television overscan.

Required improvements:

- Rename the modes to user-friendly labels:
  - `YouTube web safe`
  - `Standard`
  - `TV / overscan safe`
  - `Custom`
- Show a visual safe-zone overlay in preview, with clearly labeled boundaries.

- Use relative units or resolution-aware calculations internally, not hardcoded pixel values in components.
- Allow a custom bottom offset within validated minimum and maximum bounds.
- Add a “Check safe zone” mode that overlays a simulated YouTube control layer.
- Warn when captions overlap the safe boundary, but do not silently move user content without explanation.
- Preserve the existing centralized safe-zone constant file, but extend it to include named modes, validation ranges, and preview overlay metadata.

Implementation instruction for Antigravity: retain `lib/constants/videoSafeZones.ts` as the single source of truth. Add typed safe-zone definitions and ensure both the web preview and Remotion composition resolve positions from the same constants.

Preview experience

The “Live Cinema Preview” needs to become a real editing surface, not merely a passive video player.

Desktop preview

Required preview features:

- 16:9 preview canvas that scales responsively without changing caption layout proportions.
- Play, pause, seek, restart, mute/unmute, volume control, and playback-speed control.
- Timeline with caption segment markers.
- Current-time and total-duration display.
- Frame-by-frame stepping for precise timing review.
- Keyboard shortcuts: space for play/pause, arrow keys for seek, `J`/`K`/`L` where appropriate, and accessible focus order.
- Toggle for safe-zone overlay.
- Toggle for caption visibility.
- Toggle for audio-only waveform view when the source is audio.
- “Preview from current time” and “Preview caption segment” actions.

- A clear distinction between browser preview and final cloud render; do not promise pixel-perfect parity without validating fonts, animation, and encoding.

Implementation instruction for Antigravity: use a shared preview renderer that consumes the same caption timing and style data as the Remotion composition. Where exact parity is not feasible, label the preview as “approximate” and provide a low-resolution cloud preview render before final export.

Mobile preview

Mobile needs its own layout, not merely compressed desktop components.

Required mobile behavior:

- Use a vertical flow: preview first, then contextual controls.
- Make the preview full-width with a 16:9 aspect-ratio container.
- Use bottom sheets for transcript editing, style selection, music, and safe-zone controls.
- Use large touch targets, minimum 44 × 44 CSS pixels, for all primary actions.
- Avoid horizontal scrolling for primary controls.
- Provide a persistent bottom action bar with the next primary action, such as `Review transcript` or `Generate subtitles`.
- Collapse advanced controls behind clearly labeled sections.
- Ensure the on-screen keyboard does not cover the transcript input or action bar.
- Support orientation changes without losing state.
- Optimize preview performance: use adaptive bitrate or poster-frame previews on low-powered devices; do not attempt full desktop-grade playback on every mobile device.
- Provide a desktop recommendation for final precision editing, but do not make mobile unusable.

Implementation instruction for Antigravity: implement mobile-first responsive breakpoints and test at 360 px, 390 px, 414 px, 768 px, 1024 px, 1280 px, and 1920 px widths. Ensure no horizontal page scroll at 360 px width.

Technical architecture gaps

Render job lifecycle

The specification says AWS Lambda rendering is used, but it does not define the job lifecycle. This creates ambiguity around status, billing, retries, and user communication.

Required job states:

- `queued`
- `validating`
- `transcribing`
- `transcription_review`
- `preparing_render`
- `rendering`
- `finalizing`
- `completed`
- `failed`
- `cancelled`
- `expired`

Required job behavior:

- Create an idempotent render request using a client-generated request ID or project-version ID.
- Persist the render job in a durable database, not only in Lambda memory.
- Poll or stream progress using a secure job-status endpoint or websocket connection.
- Show meaningful progress: queued, preparing assets, rendering percentage, finalizing, and ready.
- Support user cancellation before render completion where technically feasible.
- On failure, preserve the project and transcript; never force the user to restart from upload.
- Retry transient infrastructure failures automatically with a bounded retry policy.
- Use Remotion's retry mechanism for failed render chunks, and configure retries explicitly rather than relying on defaults.
- Store render logs, error codes, input version, output URL, duration, and cost metadata for support and debugging.
- Expire or lifecycle-manage output files according to a defined retention policy.

Implementation instruction for Antigravity: implement a render orchestration service with a database-backed job record. The frontend must never treat an HTTP timeout as a definitive render failure; it must query job status by render ID.

Lambda configuration

The spec's reference to AWS Lambda timeout is incomplete. Remotion recommends validating memory, timeout, and concurrency settings; its default Lambda timeout is 120 seconds, and its concurrency model uses renderer invocations based on chunks.

Required implementation guidance:

- Set explicit Lambda memory, timeout, and concurrency values after load testing; do not rely on defaults.
- Use Remotion's recommended defaults for `framesPerLambda` unless profiling proves a custom value is better; Remotion specifically recommends leaving it unset so it can choose a reasonable value.
- Keep the maximum concurrency within Remotion's supported limit of 200 functions per render.
- Monitor render duration, cold starts, error rate, retry count, memory usage, and cost per render.
- Add a queue or workflow layer if render demand can exceed available concurrency.
- Separate transcription, render orchestration, and asset-processing workloads so a transcription delay does not consume render capacity.
- Use signed URLs and short-lived credentials for all media access.
- Add structured logging with request ID, user ID where appropriate, project ID, render ID, and error classification.

Implementation instruction for Antigravity: do not hardcode "AWS Lambda timeout" as the only failure condition. Treat timeout, throttling, missing assets, invalid input, Remotion bundle errors, storage errors, and network failures as distinct failure classes with distinct recovery paths.

Performance and scalability

Required improvements:

- Cache transcription results by content hash and user consent policy, so re-uploading the same file does not necessarily require a new transcription.
- Cache preset assets, fonts, and music metadata at the edge.
- Lazy-load the studio, transcript editor, preview player, and preset gallery.
- Use skeleton loaders and progressive disclosure rather than blocking the entire page during transcription.
- Compress and normalize preview assets separately from final render assets.
- Set explicit performance budgets:
    - Initial studio load: under 2.5 seconds on a mid-range connection.
    - Interaction response: under 100 ms for local UI actions.
    - Preview seek response: under 300 ms.
    - Render status update: near real time, with graceful fallback polling.
- Support concurrent users without allowing one large render to starve others.
- Add rate limits for uploads, transcription requests, render requests, and downloads.

Implementation instruction for Antigravity: add performance instrumentation from day one: Core Web Vitals, API latency, transcription duration, render duration, failure rate, retry rate, and credit-refund rate.

Business rules and credits

The current billing language is internally inconsistent and commercially risky.

Issues

- It says the feature is “FREE - 5 Daily Videos (0 Credits)” but also says “Paid plan exports are 100% watermark-free,” implying free exports may contain a watermark.
- It does not define what happens after five daily videos.
- It does not define whether the five-video limit is per calendar day, rolling 24 hours, or UTC-based.
- It does not state whether failed renders consume a credit or daily quota.
- It does not define whether re-rendering the same project consumes another quota.
- It does not define watermark placement, visibility, or removal conditions.

- It does not define what happens if a user changes the transcript after a successful render.

Required rules

Rule	Required definition
Daily limit	Define whether it is five successful exports per UTC day, local day, or rolling 24-hour period
Free watermark	State exact watermark text, position, opacity, and whether it can be removed
Paid export	Define which plans remove the watermark and whether credits are consumed
Failed render	Do not consume quota or credits for platform-caused failures
User-cancelled render	Define whether quota is consumed before rendering begins
Re-render	Define whether styling-only changes consume a new export
Transcript-only changes	Define whether a new render is required and whether it consumes quota

Download	Define download limits, file retention, and re-download access
Abuse prevention	Define rate limits, duplicate-upload detection, and quota abuse controls

Implementation instruction for Antigravity: centralize quota and credit logic in a server-side entitlement service. The UI must display remaining quota, next reset time, and the consequence of rendering before the user clicks `Generate`.

Error handling and edge cases

The current error handling covers only no speech, scrubber collision, and render timeout. It needs a complete recovery model.

Required error taxonomy

Error	User message direction	System behavior	Recovery
Unsupported file type	Explain accepted formats	Block upload	Let user choose another file
File too large	State limit and file size	Block upload	Suggest trimming or compressing

Video longer than 5 minutes	State maximum duration	Block or offer trim	Offer trim-to-first-5-minutes if appropriate
No audio track	Explain audio requirement	Block transcription	Ask user to upload audio-enabled media
No speech detected	Explain likely causes	No credit deduction	Let user upload another file or adjust audio
Transcription failed	Non-technical explanation	Retry automatically where appropriate	Retry, change provider, or re-upload
Low-confidence transcript	Encourage review	Preserve transcript	Let user edit and continue
Invalid caption timings	Explain issue	Block render	Highlight affected segment
Missing font or asset	Internal alert, user-friendly render error	Fail safely	Retry or use fallback font

Music asset unavailable	Explain track issue	Remove or replace selection	Let user choose another track
Render queued too long	Show expected wait and status	Keep job active	Notify when complete
Render failed	Explain failure without exposing internals	Refund quota/credits if platform-caused	Retry or contact support
Render partially completed	Explain that output is unavailable	Clean partial artifacts	Retry from render stage
Download expired	Explain retention policy	Preserve project where possible	Re-render or restore artifact
Network lost during editing	Preserve local draft state	Autosave when reconnected	Resume from saved state

Implementation instruction for Antigravity: every error must have four fields: user-facing title, user-facing explanation, recommended action, and support/debug correlation ID. Never expose stack traces, AWS errors, provider names, or internal service identifiers in the UI.

Accessibility requirements

The specification currently has no accessibility section, which is a major omission for a creator tool.

Studio accessibility

Required requirements:

- Meet WCAG 2.2 Level AA for perceivable, operable, understandable, and robust experiences.
- Ensure all interactive controls have visible focus states.
- Ensure color is not the only indicator of state, error, success, or low confidence.
- Provide accessible names for icon-only buttons.
- Maintain a logical heading hierarchy.
- Ensure modals, drawers, and bottom sheets trap focus appropriately and support Escape where appropriate.
- Ensure the transcript editor is keyboard navigable.
- Provide screen-reader announcements for upload progress, transcription completion, render status, and errors.
- Respect `prefers-reduced-motion` by reducing or disabling nonessential animation.
- Ensure text and controls meet contrast requirements; WCAG requires at least 4.5:1 for normal text and 3:1 for large text and meaningful graphical elements.

Caption accessibility

Required requirements:

- Avoid captions that obscure essential visual information.
- Ensure caption text has sufficient contrast against the video background.
- Provide readable line lengths and avoid overly long captions.
- Do not rely on animation alone to communicate meaning.
- Provide an accessible transcript in the product interface, not only burned into the video.
- Offer downloadable SRT/VTT so creators can also supply platform-native captions.

Implementation instruction for Antigravity: add automated accessibility tests for the studio UI and manual audits for the caption presets. Include keyboard-only, screen-reader, reduced-motion, zoom, and high-contrast test cases.

QA and testing plan

The current QA section is inadequate for a production video product.

Required test coverage

Area	Required tests
Unit	Caption data transformations, timing validation, safe-zone calculations, quota logic, error mapping
Integration	Upload, transcription callback, transcript save, render request, job status, artifact delivery
E2E	Full happy path from upload to download; edit transcript then render; change style then render
Visual regression	All 32 presets, safe-zone modes, font sizes, casing modes, and mobile breakpoints
Performance	Upload, transcription, preview load, render throughput, concurrent render load
Reliability	Lambda timeout, throttling, storage failure, transcription provider failure, network interruption

Security	Authorization, signed URL expiry, file-type validation, malicious filename handling, quota bypass
Accessibility	Keyboard navigation, screen reader, contrast, reduced motion, focus management
Mobile	Touch targets, keyboard overlap, orientation changes, small viewport layouts, performance
Localization	Hinglish, mixed-script text, punctuation, emoji, right-to-left text if supported
Data integrity	Transcript edits survive refresh, render retries, and provider fallback

Critical edge cases

- A video exactly 300 seconds long.
- A video one frame beyond 300 seconds.
- Audio with no detectable speech.
- Multiple speakers talking simultaneously.
- Code-switching between English and Hinglish.
- Proper nouns, numbers, acronyms, and technical terminology.
- Very fast speech and very slow speech.
- Silence at the beginning or end.
- Background noise, music, or poor-quality audio.
- A user editing a transcript after preview but before render.
- A user changing preset after preview.
- A user refreshing during transcription or render.
- Two tabs editing the same draft simultaneously.

- A render failing after credits or quota have been reserved.
- A user attempting to download an expired output.
- A user uploading an audio-only file.
- A user uploading a vertical or square video.

Implementation instruction for Antigravity: implement automated E2E coverage for the top ten user journeys and add synthetic monitoring for upload, transcription, render creation, and download availability.

UI/UX redesign recommendations

Desktop layout

The current three-row structure—stepper, preview, sticky action bar—is reasonable, but it needs clearer hierarchy and better use of space.

Recommended desktop layout:

- Left: contextual step navigation and project status.
- Center: large preview canvas and timeline.
- Right: contextual inspector for the active step.
- Bottom: persistent primary action bar with quota, estimated render time, and primary CTA.
- Top: project name, autosave status, help, and account/quota indicator.

Required improvements:

- Show a persistent progress indicator that communicates both step position and completion state.
- Make the primary CTA context-aware: `Upload media`, `Review transcript`, `Choose style`, `Preview`, or `Generate subtitles`.
- Disable—not hide—future steps until prerequisites are complete, with an explanation of what is missing.
- Use inline validation near the relevant field, not only global toasts.
- Show estimated render duration only when it can be calculated reliably; otherwise show “Usually ready in a few minutes.”
- Avoid modal-heavy flows; use drawers or inline panels for editing.

- Ensure the preview remains visible while users adjust style settings.

Mobile layout

Recommended mobile layout:

- Sticky top bar: back, project name, save status, and help.
- Full-width preview.
- Collapsible control sections.
- Bottom sheet for transcript editing and style selection.
- Sticky bottom action bar with primary CTA and remaining quota.
- Full-screen transcript editor with a prominent `Done` action.
- Full-screen preset gallery with category tabs and search.

Required improvements:

- Do not require horizontal scrolling to reach primary actions.
- Ensure the render CTA remains reachable without scrolling.
- Use progressive disclosure for advanced controls.
- Optimize the transcript editor for touch: larger tap targets, clear cursor placement, and no accidental segment deletion.
- Provide haptic or visual confirmation for destructive actions.
- Test with one-handed use and common mobile keyboards.

Implementation instructions for Antigravity

Implement the following in priority order. These instructions are intended to be directly actionable within the existing Itnavideo architecture.

Priority 1: Make the workflow reliable

1. Add a persistent `SubtitleProject` entity with a stable project ID, owner, media reference, transcript version, style configuration, render history, and status.
2. Implement autosave with optimistic UI and server reconciliation.
3. Add resume capability to `/dashboard/youtube-subtitles`.
4. Create a project list or recent-drafts entry point.
5. Ensure transcript edits are persisted separately from raw ASR output.
6. Add explicit transcript review and lock state before render.

Priority 2: Build a real preview

1. Implement a synchronized preview player with play, pause, seek, timeline, caption markers, and frame stepping.
2. Render captions in preview using the same normalized caption/style data consumed by the Remotion composition.
3. Add a safe-zone overlay with labels and warnings.
4. Add a clear “browser preview is approximate; final render is authoritative” notice where necessary.
5. Add a low-resolution cloud preview option before final render if exact parity cannot be guaranteed locally.

Priority 3: Harden rendering

1. Add a durable render-job state machine.
2. Add idempotency keys to render creation.
3. Implement secure progress polling or streaming.
4. Add bounded automatic retries for transient failures.
5. Preserve project state and transcript on failure.
6. Refund quota or credits only for platform-caused failures, according to the finalized business rule.
7. Add structured logs, error codes, and support correlation IDs.
8. Add render cancellation where feasible and clearly communicate non-cancellable stages.

Priority 4: Improve caption quality

1. Add a complete caption style schema: font, size, color, background, position, alignment, line height, letter spacing, animation, emphasis, and safe-zone offset.
2. Add contrast validation and accessible preset recommendations.
3. Add preset search, categories, favorites, and consistent previews.
4. Add custom safe-zone positioning with validation.
5. Ensure all positioning derives from `lib/constants/videoSafeZones.ts`.
6. Add downloadable SRT/VTT export from the approved transcript.

Priority 5: Improve mobile and accessibility

1. Implement mobile-first responsive layouts at 360 px and above.
2. Use bottom sheets for editing and configuration.
3. Ensure 44 × 44 CSS pixel minimum touch targets.

4. Prevent keyboard overlap with inputs and action bars.
5. Add keyboard navigation, visible focus, ARIA labels, and screen-reader announcements.
6. Respect reduced-motion preferences.
7. Add accessibility tests to CI.

Priority 6: Complete QA and observability

1. Add unit, integration, E2E, visual-regression, performance, accessibility, and failure-injection tests.
2. Add monitoring for render success rate, duration, retry rate, transcription latency, quota errors, and refund rate.
3. Add alerting for render failure spikes, transcription provider outages, and storage failures.
4. Add a support-debug view containing non-sensitive project, render, and error metadata.

Recommended specification additions

Add the following sections to the specification before implementation:

- **Project and draft lifecycle**
- **Transcript data contract**
- **Caption style schema**
- **Preview parity policy**
- **Render job state machine**
- **Credit, quota, watermark, and refund matrix**
- **Media validation matrix**
- **Error taxonomy and recovery paths**
- **Mobile interaction specification**
- **Accessibility conformance target**
- **Performance budgets**
- **Observability and alerting**
- **Data retention and deletion policy**
- **Content safety, copyright, and moderation policy**
- **Localization and language-support matrix**

Final verdict

The specification describes an attractive, focused product, but it is not yet implementation-ready for a high-quality AI video platform. Its biggest risks are an undefined transcript-editing contract, a passive preview, incomplete render reliability, unclear billing rules, and absent mobile/accessibility requirements.

The highest-leverage improvements are:

- Persistent, resumable projects.
- A robust transcript-editing and timing model.
- A frame-aware preview with safe-zone visualization.
- A durable, observable render pipeline with retries and recovery.
- Clear quota, watermark, refund, and re-render rules.
- Mobile-first, accessible interaction design.
- Comprehensive QA coverage for real-world media and failure conditions.

Implementing these changes will turn the current feature from a basic subtitle generator into a dependable, professional creator studio suitable for YouTube, podcast, education, and enterprise use. Remotion's own guidance emphasizes careful configuration of Lambda concurrency, timeout, memory, and retry behavior, which makes the missing render-orchestration requirements especially important.